arXiv:2306.03083v1 [cs.RO] 5 Jun 2023

MotionDiffuser: Controllable Multi-Agent Motion Prediction using Diffusion

Chiyu "Max" Jiang* Andre Cornman* Cheolho Park
Ben Sapp Yin Zhou Dragomir Anguelov
* equal contribution
Waymo LLC

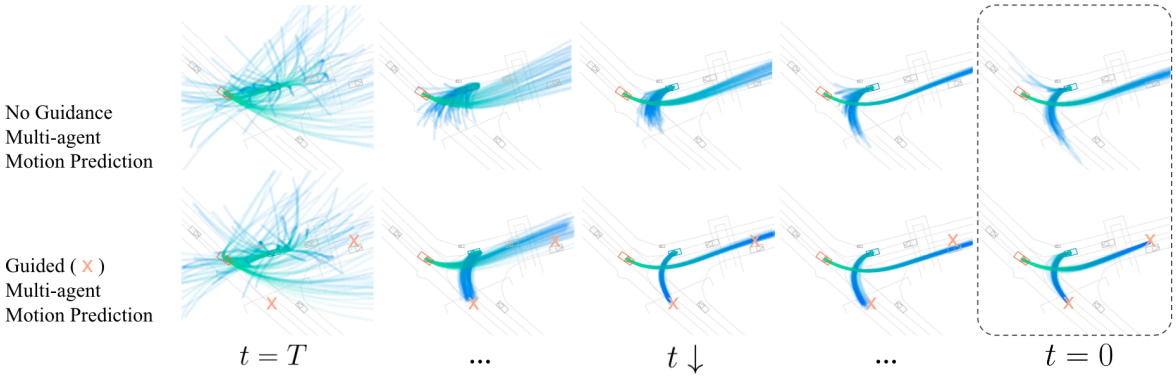


Figure 1. MotionDiffuser is a learned representation for the distribution of multi-agent trajectories based on diffusion models. During inference, samples from the predicted joint future distribution are first drawn i.i.d. from a random normal distribution (leftmost column), and gradually denoised using a learned denoiser into the final predictions (rightmost column). Diffusion allows us to learn a diverse, multimodal distribution over joint outputs (top right). Furthermore, guidance in the form of a differentiable cost function can be applied at inference time to obtain results satisfying additional priors and constraints (bottom right).

Abstract

We present MotionDiffuser, a diffusion based representation for the joint distribution of future trajectories over multiple agents. Such representation has several key advantages: first, our model learns a highly multimodal distribution that captures diverse future outcomes. Second, the simple predictor design requires only a single L2 loss training objective, and does not depend on trajectory anchors. Third, our model is capable of learning the joint distribution for the motion of multiple agents in a permutation-invariant manner. Furthermore, we utilize a compressed trajectory representation via PCA, which improves model performance and allows for efficient computation of the exact sample log probability. Subsequently, we propose a general constrained sampling framework that enables controlled trajectory sampling based on differentiable cost functions. This strategy enables a host of applications such as enforcing rules and physical priors, or creating tailored simulation scenarios. MotionDiffuser can be combined with existing backbone architectures to achieve top motion forecasting results. We obtain state-of-the-art results for multi-agent motion prediction on the Waymo Open Motion Dataset.

1. Introduction

Motion prediction is a central yet challenging problem for autonomous vehicles to safely navigate under uncertainties. Motion prediction, in the autonomous driving setting, refers to the prediction of the future trajectories of modeled agents, conditioned on the histories of the modeled agents, context agents, road graph and traffic light signals.

Several key challenges arise in the motion prediction problem. First, motion prediction is probabilistic and multimodal in nature where it is important to faithfully predict an unbiased distribution of possible futures. Second, motion prediction requires jointly reasoning about the future distribution for a set of agents that may interact with each other in each such futures. Naively predicting and sampling from the marginal distribution of trajectories for each agent independently leads to unrealistic and often conflicting outcomes. Last but not least, while it is challenging to constrain or bias the predictions of conventional regression-based trajectory models, guided sampling of the trajectories is often required. For example, it may be useful to enforce rules or physical priors for creating tailored simulation scenarios. This requires the ability to enforce constraints over the future time steps, or enforce a specified behavior for one or

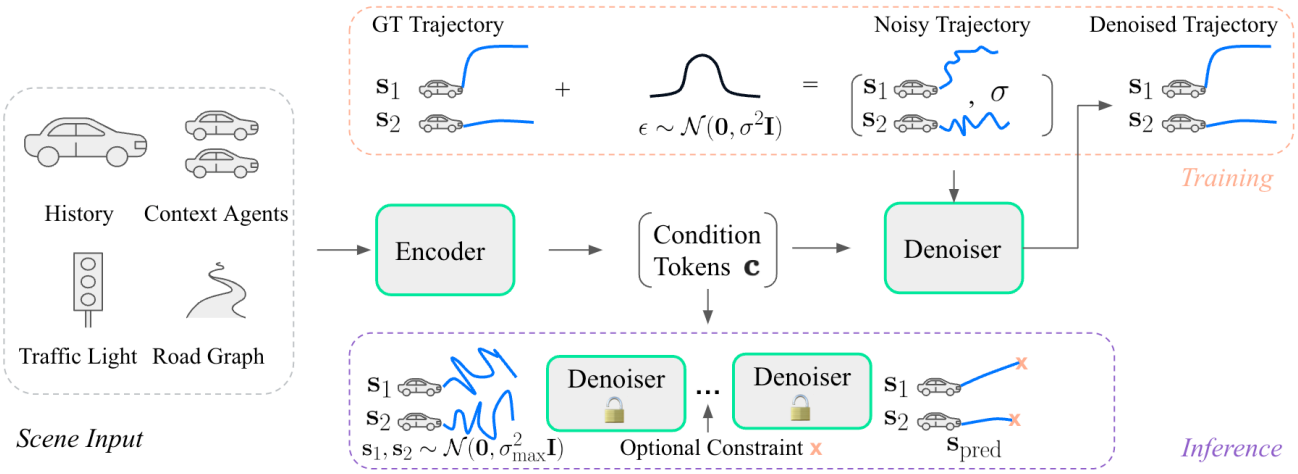


Figure 2. Overview for multi-agent motion prediction using diffusion models. The input scene containing agent history, traffic lights and road graphs is encoded via a transformer encoder into a set of condition tokens $\boldsymbol{C}$. During training, a random set of noises are sampled i.i.d. from a normal distribution and added to the ground truth (GT) trajectory. The denoiser, while attending to the condition tokens, predicts the denoised trajectories corresponding to each agent. The entire model can be trained end-to-end using a simple L2 loss between the predicted denoised trajectory and the GT trajectory. During inference, a population of trajectories for each agent can first be sampled from pure noise at the highest noise level σ_{max}, and iteratively denoised by the denoiser to produce a plausible distribution of future trajectories. An optional constraint in the form of an arbitrary differentiable loss function can be injected in the denoising process to enforce constraints.

more agents among a set of agents.

In light of these challenges, we present MotionDiffuser, a denoising diffusion model-based representation for the joint distribution of future trajectories for a set of agents (see Fig. 2). MotionDiffuser leverages a conditional denoising diffusion model. Denoising diffusion models [16, 23, 33, 43, 44] (henceforth, diffusion models) are a class of generative models that learns a denoising function based on noisy data and samples from a learned data distribution via iteratively refining a noisy sample starting from pure Gaussian noise (see Fig. 1). Diffusion models have recently gained immense popularity due to their simplicity, strong capacity to represent complex, high dimensional and multimodal distributions, ability to solve inverse problems [4, 6, 24, 44], and effectiveness across multiple problem domains, including image generation [36, 37, 39], video generation [15, 18, 49] and 3D shape generation [35].

Building on top of conditional diffusion models as a basis for trajectory generation, we propose several unique design improvements for the multi-agent motion prediction problem. First, we propose a cross-attention-based permutation-invariant denoiser architecture for learning the motion distribution for a set of agents regardless of their ordering. Second, we propose a general and flexible framework for performing controlled and guided trajectory sampling based on arbitrary differentiable cost functions of the trajectories, which enables several interesting applications such as rules and controls on the trajectories, trajectory in-painting and creating tailored simulation scenarios. Finally, we propose several enhancements to the representation, including PCA-based latent trajectory diffusion and improved trajectory sample clustering to further boost the performance of our model.

In summary, the main contributions of this work are:

- A novel permutation-invariant, multi-agent joint motion distribution representation using conditional diffusion models.
- A general and flexible framework for performing controlled and guided trajectory sampling based on arbitrary differentiable cost functions of the trajectories with a range of novel applications.
- Several significant enhancements to the representation, including PCA-based latent trajectory diffusion formulation and improved trajectory sample clustering algorithm to further boost the model performance.

2. Related Work

Denoising diffusion models Denoising diffusion models [16, 33], methodologically highly related to the class of score-based generative models [23, 43, 44], have recently emerged as a powerful class of generative models that demonstrate high sample quality across a wide range of application domains, including image generation [36, 37, 39], video generation [15, 18, 49] and 3D shape generation [35]. We are among the first to use diffusion models for predicting the joint motion of agents.

Constrained sampling Diffusion models have been shown to be effective at solving inverse problems such as image in-painting, colorization and sparse-view computed tomography by using a controllable sampling process [4–6, 22, 24, 43, 44]. Concurrent work [53] explores diffusion modeling for controllable traffic generation, which we compare to in Sec. 3.4. In diffusion models, the generation process can be conditioned on information not available during training. The inverse problem can be posed as sampling from the posterior $p(\boldsymbol{x}; \boldsymbol{y})$ based on a learned unconditional distribution $p(\boldsymbol{x})$, where $\boldsymbol{y}$ is an observation of the event $\boldsymbol{x}$. We defer further technical details to Sec. 3.4.

Motion prediction There are two main categories of approaches for motion prediction: supervised learning and generative learning. Supervised learning trains a model with logged trajectories with supervised losses such as L2 loss. One of the challenges is to model inherent multi-modal behavior of the agents. For this, MultiPath [40] uses static anchors, and MultiPath++ [48], Wayformer [31], SceneTransformer [32] use learned anchors, and DenseTNT [13] uses goal-based predictions. Home [9] and GoHome [10] predict future occupancy heatmaps, and then decode trajectories from the samples. MP3 [2] and NMP [50] learn the cost function evaluator of trajectories, and then the output trajectories are heuristically enumerated. Many of these approaches use ensembles for further diversified predictions. The next section covers generative approaches.

Generative models for motion prediction Various recent works have modeled the motion prediction task as a conditional probability inference problem of the form $p(\boldsymbol{s}; \boldsymbol{c})$ using generative models, where $\boldsymbol{s}$ denote the future trajectories of one or more agents, and $\boldsymbol{c}$ denote the context or observation. HP-GAN [1] learns a probability density function (PDF) of future human poses conditioned on previous poses using an improved Wasserstein Generative Adversarial Network (GAN). Conditional Variational Auto-Encoders (C-VAEs) [11, 20, 34], Normalizing Flows [8, 28, 29, 41] have also been shown to be effective at learning this conditional PDF of future trajectories for motion prediction. Very recent works have started looking into diffusion models as an alternative to modeling the conditional distributions of future sequences such as human motion pose sequences [38, 52] and planning [21]. In a more relevant work, [14] the authors utilize diffusion models to model the uncertainties of pedestrian motion. As far as we are aware, we are the first to utilize diffusion models to model the multi-agent joint motion distribution.

Multi-agent motion prediction While much of the motion prediction literature has worked on predicting motions of individual agents independently, there has been some work to model the motion of multiple agents jointly. SceneTransformer [32] outputs a fixed set of joint motion predictions for all the agents in the scene. M2I [45], WIMP [25], PIP [42], and CBP [47] propose a conditional model where the motions of the other agents are predicted by given motions of the controlled agents.

There is a set of literature using probabilistic graphical models. DSDNet [51] and MFP [46] use fully connected graphs. JFP [27] supports static graphs such as fully connected graphs and autonomous vehicle centered graphs, and dynamic graphs where the edges are constructed between the interacting agents. RAIN [26] learns the dynamic graph of the interaction through separate RL training.

3. Method

3.1. Diffusion Model Preliminaries

Preliminaries Diffusion models [23] provide a learned parameterization of the probability distribution $p_{\boldsymbol{\theta}}(\boldsymbol{x})$ through learnable parameters $\boldsymbol{\theta}$. Denote this probability density function, convolved with a Gaussian kernel of standard deviation σ to be $p_{\boldsymbol{\theta}}(\boldsymbol{x}, \sigma)$. Instead of directly learning a normalized probability density function $p_{\boldsymbol{\theta}}(\boldsymbol{x})$ where the normalization constant is generally intractable [19], diffusion models learn the score function of the distribution: $\nabla_{\boldsymbol{x}} \log p_{\boldsymbol{\theta}}(\boldsymbol{x}; \sigma)$ at a range of noise levels σ.

Given the score function $\nabla_{\boldsymbol{x}} \log p_{\boldsymbol{\theta}}(\boldsymbol{x}; \sigma)$, one can sample from the distribution by denoising a noise sample. Samples can be drawn from the underlying distribution $\boldsymbol{x}_0 \sim p_{\boldsymbol{\theta}}(\boldsymbol{x})$ via the following dynamics:

$$\boldsymbol{x}_0 = \boldsymbol{x}(T) + \int_T^0 -\dot{\sigma}(t)\sigma(t)\nabla_{\boldsymbol{x}} \log p_{\boldsymbol{\theta}}(\boldsymbol{x}(t); \sigma(t))dt \quad \text{where} \quad \boldsymbol{x}(T) \sim \mathcal{N}(\mathbf{0}, \sigma_{\text{max}}^2 \boldsymbol{I}) \tag{1}$$

where variance $\sigma(t)$ is a monotonic, deterministic function of an auxiliary parameter of time t. Following [23], we use the linear noise schedule $\sigma(t) = t$. The initial noise sample is sampled i.i.d. from a unit Gaussian scaled to the highest standard deviation $\sigma(T) = \sigma_{\text{max}}$.

The diffusion model can be trained to approximate a data distribution $p_\chi(\boldsymbol{x})$, where $\chi = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \cdots, \boldsymbol{x}_{Nd}\}$ denote the set of training data. The empirical distribution of the data can be viewed as a sum of delta functions around each data point: $p_\chi(\boldsymbol{x}) = \frac{1}{n}\sum_{i=0}^{N_d} \delta(\boldsymbol{x} - \boldsymbol{x}_i)$. Denote the denoiser as $D(\boldsymbol{x}; \sigma)$ which is a function that recovers the unnoised sample corresponding to the noised sample $\boldsymbol{x}$. The denoiser is related to the score function via:

$$\nabla_{\boldsymbol{x}} \log p(\boldsymbol{x}; \sigma) = (D(\boldsymbol{x}; \sigma) - \boldsymbol{x})/\sigma^2 \tag{2}$$

The denoiser can be learned by minimizing the expected L_2 denoising error for a perturbed sample $\boldsymbol{x}$ at any noise level σ sampled from the noise distribution $q(\sigma)$:

$$\arg\min_{\theta} \; \mathbb{E}_{\boldsymbol{x}\sim p_\chi} \mathbb{E}_{\sigma\sim q(\sigma)} \mathbb{E}_{\boldsymbol{\epsilon}\sim\mathcal{N}(\mathbf{0},\sigma^2\boldsymbol{I})} ||D_{\boldsymbol{\theta}}(\boldsymbol{x} + \boldsymbol{\epsilon}; \sigma) - \boldsymbol{x}||_2^2 \tag{3}$$

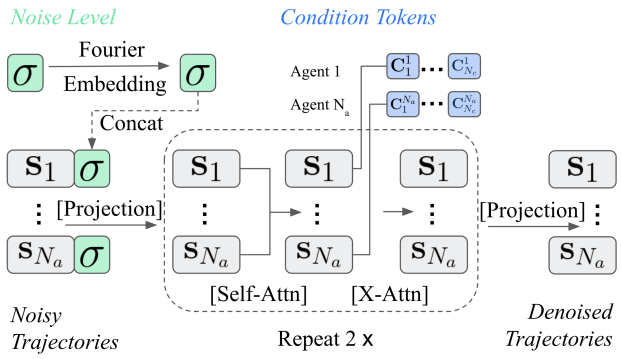


Figure 3. Network architecture for set denoiser $D_{\boldsymbol{\theta}}(\boldsymbol{S}; \boldsymbol{C}, \sigma)$. The noisy trajectories corresponding to agents $\boldsymbol{s}_1 \cdots \boldsymbol{s}_{N_a}$ are first concatenated with a random-fourier encoded noise level σ, before going through repeated blocks of self-attention among the set of trajectories and cross-attention with respect to the condition tokens $\boldsymbol{c}_1 \cdots \boldsymbol{c}_{N_c}$. The self-attention allows the diffusion model to learn a joint distribution across the agents and cross-attention allows the model to learn a more accurate scene-conditional distribution. Note that each agent cross-attends to its own condition tokens from the agent-centric scene encoding (not shown for simplicity). The [learnable components] are marked with brackets.

Conditional diffusion models In this work, we are interested in the conditional setting of learning $p_{\boldsymbol{\theta}}(\boldsymbol{x}; \boldsymbol{c})$, where $\boldsymbol{x}$ denote the future trajectories of a set of agents and $\boldsymbol{c}$ is the scene context. A simple modification is to augment both the denoiser $D(\boldsymbol{x}; \boldsymbol{c}, \sigma)$ and the score function $\nabla_{\boldsymbol{x}} \log p(\boldsymbol{x}; \boldsymbol{c}, \sigma)$ by the condition $\boldsymbol{c}$. Given a dataset χ_c augmented by conditions: $\chi_c = \{(\boldsymbol{x}_1, \boldsymbol{c}_1), \cdots, (\boldsymbol{x}_{N_d}, \boldsymbol{c}_N)\}$, the conditional denoiser can be learned by a conditional denoising score matching objective to minimize the following:

$$\mathbb{E}_{\boldsymbol{x}, \boldsymbol{c} \sim \chi_c} \mathbb{E}_{\sigma \sim q(\sigma)} \mathbb{E}_{\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \boldsymbol{I})} ||D_{\boldsymbol{\theta}}(\boldsymbol{x} + \boldsymbol{\epsilon}; \boldsymbol{c}, \sigma) - \boldsymbol{x}||_2^2 \tag{4}$$

which leads to the learned conditional score function:

$$\nabla_{\boldsymbol{x}} \log p_{\boldsymbol{\theta}}(\boldsymbol{x}; \boldsymbol{c}, \sigma) = (D_{\boldsymbol{\theta}}(\boldsymbol{x}; \boldsymbol{c}, \sigma) - \boldsymbol{x})/\sigma^2 \tag{5}$$

Preconditioning and training Directly training the model with the denoising score matching objective (Eqn. 4) has various drawbacks. First, the input to the denoiser has non-unit variance: $\text{Var}(\boldsymbol{x} + \boldsymbol{\epsilon}) = \text{Var}(\boldsymbol{x}) + \text{Var}(\boldsymbol{\epsilon}) = \sigma_{\text{data}}^2 + \sigma^2$, $\sigma \in [0, \sigma_{\text{max}}]$. Second, at small noise levels of σ, it is much easier for the model to predict the residual noise than predicting the clean signal. Following [23], we adopt a preconditioned form of the denoiser:

$$D_{\boldsymbol{\theta}}(\boldsymbol{x}; \boldsymbol{c}, \sigma) = c_{\text{skip}}(\sigma)\boldsymbol{x} + c_{\text{out}}(\sigma) F_{\boldsymbol{\theta}}(c_{\text{in}}(\sigma)\boldsymbol{x}; \boldsymbol{c}, c_{\text{noise}}(\sigma)) \tag{6}$$

$F_{\boldsymbol{\theta}}$ is the neural network to train, $c_{\text{skip}}, c_{\text{in}}, c_{\text{out}}, c_{\text{noise}}$ respectively scale the skip connection to the noisy $\boldsymbol{x}$, input to the network, output from the network, and noise input σ to the network. We do not additionally scale $\boldsymbol{c}$ since it is the output of an encoder network, assumed to have modulated scales.

Sampling We follow the ODE dynamics in Eqn. 1 when sampling the predictions. We utilize Huen's 2nd order method for solving the corresponding ODE using the default parameters and 32 sampling steps.

3.2. Diffusion Model for Multi-Agent Trajectories

One of the main contributions of this work is to propose a framework for modeling the joint distribution of multi-agent trajectories using diffusion models. Denote the future trajectory of agent i as $\boldsymbol{s}_i \in \mathbb{R}^{N_t \times N_f}$ where N_t is the number of future time steps and N_f is the number of features per time steps, such as longitudinal and lateral positions, heading directions etc. Denote $\boldsymbol{c}_i \in \mathbb{R}^{\cdots}$ as the learned ego-centric context encoding of the scene, including the road graph, traffic lights, histories of modeled and context agents, as well as interactions within these scene elements, centered around agent i. For generality $\boldsymbol{c}$ could be of arbitrary dimensions, either as a single condition vector, or as a set of context tokens. Denote the set of agent futures trajectories as $\boldsymbol{S} \in \mathbb{R}^{N_a \times N_t \times N_f}$, the set of ego-centric context encodings as $\boldsymbol{C} \in \mathbb{R}^{N_a \times \cdots}$, where $|\mathcal{S}| = |\mathcal{C}| = N_a$ is the number of modeled agents. We append each agent's position and heading (relative to the ego vehicle) to its corresponding context vectors.Denote the j-th permutation of agents in the two sets to be $\mathcal{S}^j, \mathcal{C}^j$, sharing consistent ordering of the agents. We seek to model the set probability distribution of agent trajectories using diffusion models: $p(\mathcal{S}^j; \mathcal{C}^j)$. Since the agent ordering in the scene is arbitrary, learning a permutation invariant set probability distribution is essential, i.e.,

$$p(\boldsymbol{S}; \boldsymbol{C}) = p(\boldsymbol{S}^j; \boldsymbol{C}^j), \forall j \in [1, N_a!] \tag{7}$$

To learn a permutation-invariant set probability distribution, we seek to learn a permutation-*equivariant* denoiser, i.e., when the order of the agents in the denoiser permutes, the denoiser output follows the same permutation:

$$D(\boldsymbol{S}^j; \boldsymbol{C}^j, \sigma) = D^j(\boldsymbol{S}; \boldsymbol{C}, \sigma), \forall j \in [1, N_a!] \tag{8}$$

Another major consideration for the denoiser architecture is the ability to effectively attend to the condition tensor $\boldsymbol{c}$ and noise level σ. Both of these motivations prompt us to utilize the transformer as the main denoiser architecture. We utilize the scene encoder architecture from the state-of-the-art Wayformer [31] model to encode scene elements such as road graph, agent histories and traffic light states into a set of latent embeddings. The denoiser takes as input the GT trajectory corresponding to each agent, perturbed with a random noise level $\sigma \sim q(\sigma)$, and the noise level σ. During the denoising process, the noisy input undergoes repeated blocks of self-attention between the agents and cross attention to the set of context tokens per agent, and finally the results are projected to the same feature dimensionality as the inputs. Since we do not apply positional encoding along the agent dimension, transformers naturally preserve

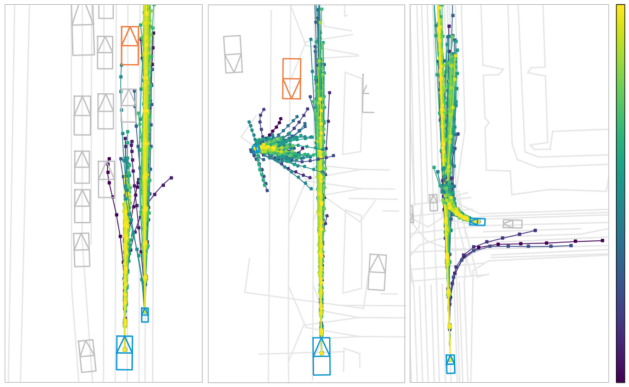


Figure 4. Inferred exact log probability of 64 sampled trajectories per agent. Higher probability samples are plotted with lighter colors. The orange agent represents the AV (autonomous vehicle).

the equivariance among the tokens (agents), leading to the permutation-equivarnance of the denoiser model. See Fig. 3 for a more detailed design of the transformer-based denoiser architecture.

3.3. Exact Log Probability Inference

With our model, we can infer the exact log probability of the generated samples with the following method. First, the change of log density over time follows a second differential equation, called the instantaneous change of variables formula [3],

$$\frac{\log p(\boldsymbol{x}(t))}{\partial t} = -Tr\left(\frac{\partial f}{\partial \boldsymbol{x}(t)}\right) \quad \text{where} f = \partial \boldsymbol{x}/\partial t \tag{9}$$

In the diffusion model, the flow function, f follows,

$$f(\boldsymbol{x}(t), t) = \frac{\partial \boldsymbol{x}(t)}{\partial t} = -\dot{\sigma}(t)\sigma(t)\nabla_{\boldsymbol{x}} \log p(\boldsymbol{x}(t); \sigma(t)) \tag{10}$$

The log probability of the sample can be calculated by integrating over time as below.

$$\log p(\boldsymbol{x}(0)) = \log p(\boldsymbol{x}(T)) - \int_T^0 Tr\left(\frac{\partial f}{\partial \boldsymbol{x}(t)}\right) dt \tag{11}$$

The computation of the trace of the Jacobian takes $O(n^2)$ where n is the dimensionality of $\boldsymbol{x}$. When we use PCA as in Sec. 3.5, n will be much smaller than the dimensionality of the original data. We can also use Hutchinson's trace estimator as in FFJORD [12] which takes $O(n)$.

The log probability can be used for filtering higher probability predictions. In Fig. 4, for example, higher probability samples plotted with lighter colors are more likely.

3.4. Constraining Trajectory Samples

Constrained trajectory sampling has a range of applications. One situation where controllability of the sampled trajectories would be required is to inject physical rules and constraints. For example, agent trajectories should avoid collision with static objects and other road users. Another application is to perform trajectory in-painting: to solve the inverse problem of completing the trajectory prediction given one or more control points. This is a useful tool in creating custom traffic scenarios for autonomous vehicle development and simulation.

More formally, we seek the solution to sampling from the joint conditional distribution $p(\boldsymbol{S}; \boldsymbol{C}) \cdot q(\boldsymbol{S}; \boldsymbol{C})$, where $p(\boldsymbol{S}; \boldsymbol{C})$ is the learned future distribution for trajectories and $q(\boldsymbol{S}; \boldsymbol{C})$ a secondary distribution representing the constraint manifold for $\boldsymbol{S}$. The score of this joint distribution is $\nabla_{\boldsymbol{S}} \log \Big(p(\boldsymbol{S}; \boldsymbol{C}) \cdot q(\boldsymbol{S}; \boldsymbol{C})\Big) = \nabla_{\boldsymbol{S}} \log p(\boldsymbol{S}; \boldsymbol{C}) + \nabla_{\boldsymbol{S}} \log q(\boldsymbol{S}; \boldsymbol{C})$. In order to sample this joint distribution, we need the joint score function at all noise levels σ:

$$\nabla_{\boldsymbol{S}} \log p(\boldsymbol{S}; \boldsymbol{C}, \sigma) + \nabla_{\boldsymbol{S}} \log q(\boldsymbol{S}; \boldsymbol{C}, \sigma) \tag{12}$$

The first term directly corresponds to the conditional score function in Eqn. 5. The second term accounts for gradient guidance based on the constraint, which resembles classifier-based guidance [17] in class-conditional image generation tasks, where a specialty neural network is trained to estimate this guidance term under a range of noise levels. We refer to this as the constraint gradient score. However, since our goal is to approximate the constraint gradient score with an arbitrary differentiable cost function of the trajectory, how is this a function of the noise parameter σ?

The key insight is to exploit the duality between any intermediate noisy trajectory $\boldsymbol{S}$ and the denoised trajectory at that noise level $D(\boldsymbol{S}; \boldsymbol{C}, \sigma)$. While $\boldsymbol{S}$ is clearly off the data manifold and not a physical trajectory, $D(\boldsymbol{S}; \boldsymbol{C}, \sigma)$ usually closely resembles a physical trajectory that is on the data manifold since it is trained to regress for the ground truth (Eqn. 4), even at a high σ value. The denoised event and the noisy event converge at the limit $\sigma \to 0$. In this light, we approximate the constraint gradient score as:

$$\nabla_{\boldsymbol{S}} \log q(\boldsymbol{S}; \boldsymbol{C}, \sigma) \approx \lambda \frac{\partial}{\partial \boldsymbol{S}} \mathcal{L}\Big(D(\boldsymbol{S}; \boldsymbol{C}, \sigma)\Big) \tag{13}$$

where $\mathcal{L} : \mathbb{R}^{N_a \times N_t \times N_f} \mapsto \mathbb{R}$ is an arbitrary cost function for the set of sampled trajectories, and λ is a hyperparameter controlling the weight of this constraint.

In this work, we introduce two simple cost functions for trajectory controls: an attractor and a repeller. Attractors encourage the predicted trajectory at certain timesteps to arrive at certain locations. Repellers discourage interacting agents from getting too close to each other and mitigates collisions. We define the costs as:

Attractor cost

$$\mathcal{L}_{\text{attract}}(D(\boldsymbol{S};\boldsymbol{C},\sigma)) = \frac{\sum|(D(\boldsymbol{S};\boldsymbol{C},\sigma) - \boldsymbol{S}_{\text{target}}) \odot \boldsymbol{M}_{\text{target}}|}{\sum|\boldsymbol{M}_{\text{target}}| + eps} \tag{14}$$

Where $\boldsymbol{S}_{\text{target}} \in \mathbb{R}^{N_a \times N_t \times N_f}$ are the target location tensor, and $\boldsymbol{M}_{\text{target}}$ is a binary mask tensor indicating which locations in $\boldsymbol{S}_{\text{target}}$ to enforce. $\odot$ denotes the elementwise product and eps denotes an infinitesimal value to prevent underflow.

Repeller cost

$$\boldsymbol{A} = \texttt{max}\Big(\big(1 - \frac{1}{r}\Delta(D(\boldsymbol{S};\boldsymbol{C},\sigma))\big) \odot (1 - I), 0\Big) \tag{15}$$

$$\mathcal{L}_{\text{repell}}(D(\boldsymbol{S})) = \frac{\sum \boldsymbol{A}}{\sum(\boldsymbol{A} > 0) + eps} \tag{16}$$

Where A is the per time step repeller cost. we denote the pairwise L_2 distance function between all pairs of denoised agents at all time steps as $\Delta(D(\boldsymbol{S};\boldsymbol{C},\sigma)) \in \mathbb{R}^{N_a \times N_a \times N_t}$, identity tensor broadcast to all N_t time steps $\boldsymbol{I} \in \mathbb{R}^{N_a \times N_a \times N_t}$, and repeller radius as r.

Constraint score thresholding To further increase the stability of the constrained sampling process, we propose a simple and effective strategy: constraint score thresholding (ST). From Eqn. 2, we make the observation that:

$$\sigma\nabla_{\boldsymbol{x}} \log p(\boldsymbol{x};\sigma) = (D(\boldsymbol{x},\sigma) - \boldsymbol{x})/\sigma = \boldsymbol{\epsilon},\ \boldsymbol{\epsilon} \sim \mathcal{N}(\boldsymbol{0},\boldsymbol{I}) \tag{17}$$

Therefore, we adjust the constraint score in Eqn. 13 via an elementwise clipping function:

$$\nabla_{\boldsymbol{S}} \log q(\boldsymbol{S};\boldsymbol{C},\sigma) := \texttt{clip}(\sigma\nabla_{\boldsymbol{S}} \log q(\boldsymbol{S};\boldsymbol{C},\sigma), \pm 1)/\sigma \tag{18}$$

We ablate this design choice in Table 2.

3.5. Trajectory Representation Enhancements

Sample clustering While MotionDiffuser learns an entire distribution of possible joint future trajectories from which we can draw an arbitrary number of samples, it is often necessary to extract a more limited number of representative modes from the output distribution. The Interaction Prediction challenge in Waymo Open Motion Dataset, for instance, computes metrics based on a set of 6 predicted joint futures across modeled agents. Thus, we need to generate a representative set from the larger set of sampled trajectories.

To this end, we follow the trajectory aggregation method defined in [48] which performs iterative greedy clustering to maximize the probability of trajectory samples falling within a fixed distance threshold to an output cluster. We refer readers to [48] for details on the clustering algorithm.

In the joint agent prediction setting, we modify the clustering algorithm such that for each joint prediction sample, we maximize the probability that all agent predictions fall within a distance threshold to an output cluster.

PCA latent diffusion Inspired by the recent success of latent diffusion models [37] for image generation, we utilize a compressed representation for trajectories using Principal Component Analysis (PCA). PCA is particularly suitable for representing trajectories, as trajectories are temporally and geometrically smooth in nature, and the trajectories can be represented by a very small set of components. Our analysis shows that a mere 3 components (for trajectories with 80×2 degrees of freedom) accounts for 99.7% of all explained variance, though we use 10 components for a more accurate reconstruction. PCA representation has multiple benefits, including faster inference, better success with controlled trajectory, and perhaps most importantly, better accuracy and performance (see ablation studies in Sec. 5).

First, as many ground truth trajectories include missing time steps (due to occlusion / agent leaving the scene), we use linear interpolation / extrapolation to fill in the missing steps in each trajectory. We uniformly sample a large population of $N_s = 10^5$ agent trajectories, where each trajectory $\boldsymbol{s}_i \in \mathbb{R}^{N_t N_f}, i \in [1, N_s]$ is first centered around the agent's current location, rotated such that the agent's heading is in $+y$ direction, and flattened into a single vector. Denote this random subset of agent trajectories as $\boldsymbol{S}' \in \mathbb{R}^{N_s \times N_t N_f}$. We compute its corresponding principle component matrix (with whitening) as $W_{\text{pca}} \in \mathbb{R}^{N_p \times (N_t N_f)}$ where N_p is the number of principle components to use, and its mean as $\bar{\boldsymbol{s}}' \in \mathbb{R}^{N_t N_f}$. We obtain the PCA and inverse PCA transformation for each trajectory $\boldsymbol{S}_i$ as:

$$\hat{\boldsymbol{s}}_i = (\boldsymbol{s}_i - \bar{\boldsymbol{s}})W_{\text{pca}}^T \Leftrightarrow \boldsymbol{s}_i = \hat{\boldsymbol{s}}_i(W_{\text{pca}}^T)^{-1} + \bar{\boldsymbol{s}} \tag{19}$$

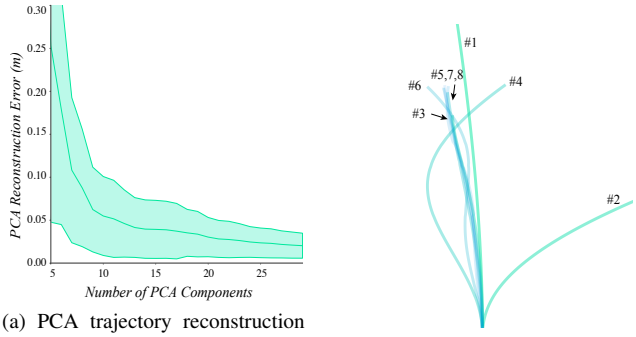
With the new representation, we have agent trajectories in Eqn. 7 in PCA space as $\boldsymbol{S} \in \mathbb{R}^{N_a \times N_p}$.

4. Experiment and Results

4.1. PCA Mode Analysis

To motivate our use of PCA as a simple and accurate compressed trajectory representation, we analyze the principal components computed from $N_s = 10^5$ randomly selected trajectories from the Waymo Open Dataset training split. Fig. 5a shows the average reconstruction error per waypoint using increasing numbers of principal components. When keeping only the first 10 principal components, the average reconstruction error is 0.06 meters, which is significantly lower than the average prediction error achieved by state-of-the-art methods. This motivates PCA as an effective compression strategy, without the need for more complex strategies like autoencoders in [37].

We visualize the top-10 principal components in Fig. 5b. The higher order principal components are increasingly similar, and deviate only slightly from the dataset mean. These components represent high frequency trajectory information that are irrelevant for modeling, and may also be a result of perception noise.



(a) PCA trajectory reconstruction error vs number of PCA components.

(b) Visualization of the top-10 PCA components for trajectories.

Figure 5. Analysis of PCA representation for agent trajectories. (a) shows the average reconstruction error for varying numbers of principal components. (b) shows a visualization of the top-10 principal components. The higher modes representing higher frequencies are increasingly similar and have a small impact on the final trajectory.

4.2. Multi-Agent Motion Prediction

To evaluate MotionDiffuser's performance in the multi-agent prediction setting, we assess our method on the Waymo Open Dataset Interactive split, which contains pairs of agents in highly interactive and diverse scenarios [7].

In Table 1, we report the main metrics for the Interactive split, as defined in [7]. minSADE measures the displacement between the ground-truth future agent trajectories and the closest joint prediction (out of 6 joint predictions), averaged over the future time horizon and over the pair of interacting agents. minSFDE measures the minimum joint displacement error at the time horizon endpoint. SMissRate measures the recall of the joint predictions, with distance thresholds defined as a function of agent speed and future timestep. Finally mAP measures the joint Mean Average Precision based on agent action types, such as left-turn and u-turn. The reported metrics are averaged over future time horizons (3s, 5s, and 8s) and over agent types (vehicles, pedestrians, and cyclists) .

Additionally, we report results for the Overlap metric [27] by measuring the overlap rate on the most likely joint prediction, which captures the consistency of model predictions, as consistent joint predictions should not collide.

Our model achieves state-of-the-art results, as shown in Table 1. While MotionDiffuser and Wayformer [31] use the same backbone, our method performs significantly better across all metrics due to the strength of the diffusion head. Compared to JFP [27] on the test split, we demonstrate an improvement with respect to the minSADE and minSFDE metrics. For mAP, and Overlap, our method performs slightly worse than JFP, but outperforms all other methods.

	Method	Overlap (↓)	minSADE (↓)	minSFDE (↓)	sMissRate (↓)	mAP (↑)
Test	LSTM baseline [7]	-	1.91	5.03	0.78	0.05
	HeatIRm4 [30]	-	1.42	3.26	0.72	0.08
	SceneTransformer(J) [32]	-	0.98	2.19	0.49	0.12
	M2I [45]	-	1.35	2.83	0.55	0.12
	DenseTNT [13]	-	1.14	2.49	0.54	0.16
	MultiPath++[48]	-	1.00	2.33	0.54	0.17
	JFP [27]	-	0.88	1.99	**0.42**	**0.21**
	MotionDiffuser (Ours)	-	**0.86**	**1.95**	0.43	0.20
Val	SceneTransformer(M) [32]	0.091	1.12	2.60	0.54	0.09
	SceneTransformer(J) [32]	0.046	0.97	2.17	0.49	0.12
	MultiPath++ [48]	0.064	1.00	2.33	0.54	0.18
	JFP [27]	**0.030**	0.87	1.96	**0.42**	**0.20**
	Wayformer [31]	0.061	0.99	2.30	0.47	0.16
	MotionDiffuser (Ours)	0.036	**0.86**	**1.92**	**0.42**	0.19

Table 1. WOMD Interactive Split: we report scene-level joint metrics numbers averaged for all object types over t = 3, 5, 8 seconds. Metrics minSADE, minSFDE, SMissRate, and mAP are from the benchmark [7]. Overlap is defined in [27].

Method	Realism (↓)			Constraint Effectiveness			
	minSADE	meanSADE	Overlap	minSFDE (↓)	meanSFDE (↓)	SR2m (↑)	SR5m (↑)
No Constraint	1.261	3.239	0.059	2.609	8.731	0.059	0.316
Attractor (to GT final point)							
Optimization	4.563	5.385	0.054	*0.010*	**0.074**	**1.000**	**1.000**
GTC[53]	1.18	**1.947**	0.057	0.515	0.838	0.921	0.957
Ours (-ST)	*1.094*	*2.083*	0.042	0.627	1.078	0.913	0.949
Ours	**0.533**	2.194	*0.040*	**0.007**	*0.747*	*0.952*	*0.994*
Repeller (between the pair of agents)							
Ours	1.359	3.229	**0.008**	2.875	8.888	0.063	0.317

Table 2. Quantitative validation for controllable trajectory synthesis. We enforce the attractor or repeller constraints in Sec. 3.4.

4.3. Controllable Trajectory Synthesis

We experimentally validate the effectiveness of our controllable trajectory synthesis approach. In particular, we validate the attractor and repeller designs proposed in Sec. 3.4. We continue these experiments using the Interactive Split from Waymo Open Motion Dataset. In experiments for both the attractor and the repeller, we use the same baseline diffusion model trained in Sec. 4.2. We randomly sample 64 trajectories from the predicted distribution. We report our results in Table 2. We measure min/mean ADE/FDE and overlap metrics, following Sec. 4.2. The mean metrics computes the mean quantity over the 64 predictions.

For the attractor experiment, we constrain the last point of all predicted trajectories to be close to the last point in the ground truth data. Therefore, min/meanSADE serves as a proxy for the realism of the predictions and how closely they stay to the data manifold. For baselines, we compare to two approaches: "Optimization" directly samples the trajectories from our diffusion model, followed by a post processing step via Adam optimizer to enforce the constraints. "CTG" is a reimplementation of the sampling method in a concurrent work [53] that performs an inner optimization loop to enforce constraints on the denoised samples during every step of the diffusion process. See Table 2 for detailed results. Although trajectory optimization after the sampling

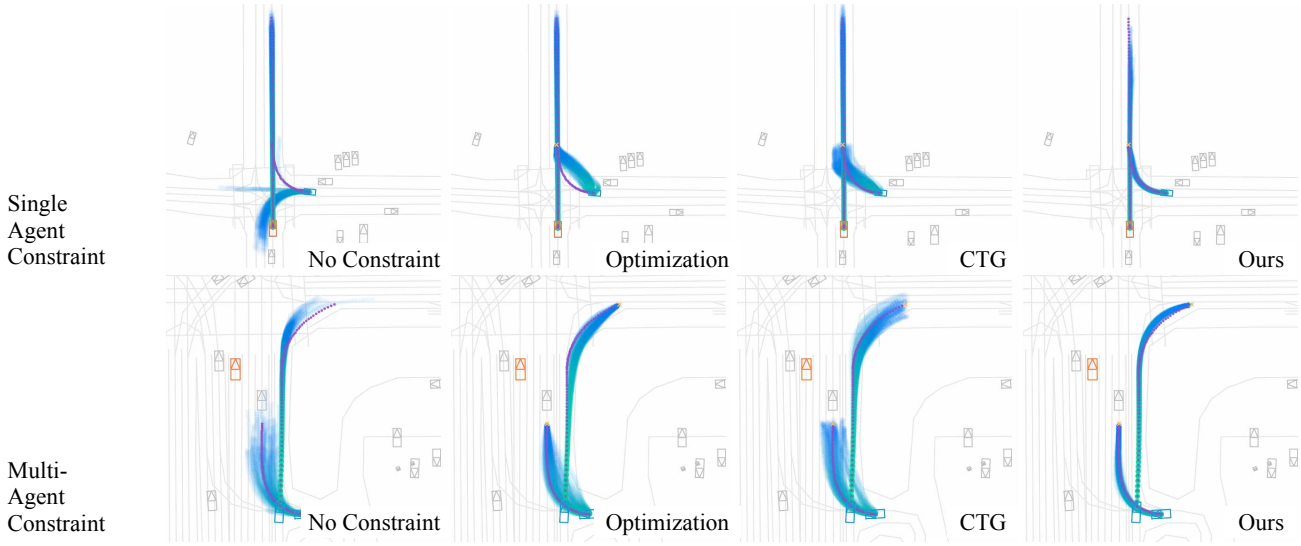


Figure 6. Qualitative results for controllable trajectory synthesis. We apply an attractor-based constraint (marked as $\times$) on the last point of the trajectory. Without any constraint at inference time, the initial prediction distributions from MotionDiffuser ("No Constraint") are plausible yet dispersed. While test time optimization of the predicted trajectories is effective at enforcing the constraints on model outputs, it deviates significantly from the data manifold, resulting in unrealistic outputs. Our method produces realistic and well-constrained results.

Method	minSADE(↓)	minSFDE(↓)	SMissRate(↓)
Ours (-PCA)	1.03	2.29	0.53
Ours (-Transformer)	0.93	2.08	0.47
Ours (-SelfAttention)	0.91	2.07	0.46
MotionDiffuser (Ours)	0.88	1.97	0.43

Table 3. Ablations on WOMD Interactive Validation Split. We ablate components of the denoiser architecture, and the PCA compressed trajectory representation.

process has the strongest effect in enforcing constraints, it results in unrealistic trajectories. With our method we have a high level of effectiveness in enforcing the trajectories, second only to optimization methods, while maintaining a high degree of realism. Additionally we show qualitative comparisons for the optimized trajectories in Fig. 6.

For the repeller experiment, we add the repeller constraint (radius $5m$) between all pairs of modeled agents. We were able to significantly decrease overlap between joint predictions by an order of magnitude, demonstrating its effectiveness in repelling between the modeled agents.

5. Ablation Studies

We validate the effectiveness of our proposed Score Thresholding (ST) approach in Table 2, with *Ours(-ST)* denoting the removal of this technique, resulting in significantly worse constraint satisfaction.

Furthermore, we ablate critical components of the MotionDiffuser architecture in Table 3. We find that using the uncompressed trajectory representation *Ours(-PCA)* degrades performance significantly. Additionally, replacing the Transformer architecture with a simple MLP *Ours(-Transformer)* reduces performance. We also ablate the self-attention layers in the denoiser architecture *Ours(-SelfAttention)*, while keeping the cross-attention layers (to allow for conditioning on the scene context and noise level). This result shows that attention between modeled agents' noisy future trajectories is important for generating consistent joint predictions. Note that MotionDiffuser's performance in Table 3 is slightly worse than Table 1 due to a reduced Wayformer encoder backbone size.

6. Conclusion and Discussions

In this work, we introduced MotionDiffuser, a novel diffusion model based multi-agent motion prediction framework that allows us to learn a diverse, multimodal joint future distribution for multiple agents. We propose a novel transformer-based set denoiser architecture that is permutation invariant across agents. Furthermore we propose a general and flexible constrained sampling framework, and demonstrate the effectiveness of two simple and useful constraints - attractor and repeller. We demonstrate state-of-the-art multi-agent motion prediction results, and the effectiveness of our approach on Waymo Open Motion Dataset.

Future work includes applying the diffusion-based generative modeling technique to other topics of interest in autonomous vehicles, such as planning and scene generation.

Acknowledgements We thank Wenjie Luo for helping with the overlap metrics code, Ari Seff for helping with multi-agent NMS, Rami Al-RFou, Charles Qi and Carlton Downey for helpful discussions, Joao Messias for reviewing the manuscript, and anonymous reviewers.

References

[1] Emad Barsoum, John Kender, and Zicheng Liu. Hp-gan: Probabilistic 3d human motion prediction via gan. In *Proceedings of the IEEE conference on computer vision and pattern recognition workshops*, pages 1418–1427, 2018. 3

[2] Sergio Casas, Abbas Sadat, and Raquel Urtasun. Mp3: A unified model to map, perceive, predict and plan. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14403–14412, 2021. 3

[3] Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary differential equations. *Advances in neural information processing systems*, 31, 2018. 5

[4] Jooyoung Choi, Sungwon Kim, Yonghyun Jeong, Youngjune Gwon, and Sungroh Yoon. Ilvr: Conditioning method for denoising diffusion probabilistic models. *arXiv preprint arXiv:2108.02938*, 2021. 2, 3

[5] Hyungjin Chung, Byeongsu Sim, Dohoon Ryu, and Jong Chul Ye. Improving diffusion models for inverse problems using manifold constraints. *arXiv preprint arXiv:2206.00941*, 2022.

[6] Hyungjin Chung, Byeongsu Sim, and Jong Chul Ye. Come-closer-diffuse-faster: Accelerating conditional diffusion models for inverse problems through stochastic contraction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12413–12422, 2022. 2, 3

[7] Scott Ettinger, Shuyang Cheng, Benjamin Caine, Chenxi Liu, Hang Zhao, Sabeek Pradhan, Yuning Chai, Ben Sapp, Charles Qi, Yin Zhou, Zoey Yang, Aurelien Chouard, Pei Sun, Jiquan Ngiam, Vijay Vasudevan, Alexander McCauley, Jonathon Shlens, and Dragomir Anguelov. Large scale interactive motion forecasting for autonomous driving: The waymo open motion dataset. *arXiv preprint arXiv:2104.10133*, 2021. 7

[8] Samuel G Fadel, Sebastian Mair, Ricardo da Silva Torres, and Ulf Brefeld. Contextual movement models based on normalizing flows. *AStA Advances in Statistical Analysis*, pages 1–22, 2021. 3

[9] Thomas Gilles, Stefano Sabatini, Dzmitry Tsishkou, Bogdan Stanciulescu, and Fabien Moutarde. Home: Heatmap output for future motion estimation. In *2021 IEEE International Intelligent Transportation Systems Conference (ITSC)*, pages 500–507. IEEE, 2021. 3

[10] Thomas Gilles, Stefano Sabatini, Dzmitry Tsishkou, Bogdan Stanciulescu, and Fabien Moutarde. Gohome: Graph-oriented heatmap output for future motion estimation. In *2022 International Conference on Robotics and Automation (ICRA)*, pages 9107–9114. IEEE, 2022. 3

[11] Sebastian Gomez-Gonzalez, Sergey Prokudin, Bernhard Schölkopf, and Jan Peters. Real time trajectory prediction using deep conditional generative models. *IEEE Robotics and Automation Letters*, 5(2):970–976, 2020. 3

[12] Will Grathwohl, Ricky TQ Chen, Jesse Bettencourt, Ilya Sutskever, and David Duvenaud. Ffjord: Free-form continuous dynamics for scalable reversible generative models. *arXiv preprint arXiv:1810.01367*, 2018. 5

[13] Junru Gu, Qiao Sun, and Hang Zhao. Densetnt: Waymo open dataset motion prediction challenge 1st place solution. *CoRR*, abs/2106.14160, 2021. 3, 7

[14] Tianpei Gu, Guangyi Chen, Junlong Li, Chunze Lin, Yongming Rao, Jie Zhou, and Jiwen Lu. Stochastic trajectory prediction via motion indeterminacy diffusion. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17113–17122, 2022. 3

[15] Jonathan Ho, William Chan, Chitwan Saharia, Jay Whang, Ruiqi Gao, Alexey Gritsenko, Diederik P Kingma, Ben Poole, Mohammad Norouzi, David J Fleet, et al. Imagen video: High definition video generation with diffusion models. *arXiv preprint arXiv:2210.02303*, 2022. 2

[16] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020. 2

[17] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*, 2022. 5

[18] Jonathan Ho, Tim Salimans, Alexey Gritsenko, William Chan, Mohammad Norouzi, and David J Fleet. Video diffusion models. *arXiv preprint arXiv:2204.03458*, 2022. 2

[19] Aapo Hyvärinen and Peter Dayan. Estimation of non-normalized statistical models by score matching. *Journal of Machine Learning Research*, 6(4), 2005. 3

[20] Boris Ivanovic, Karen Leung, Edward Schmerling, and Marco Pavone. Multimodal deep generative models for trajectory prediction: A conditional variational autoencoder approach. *IEEE Robotics and Automation Letters*, 6(2):295–302, 2020. 3

[21] Michael Janner, Yilun Du, Joshua Tenenbaum, and Sergey Levine. Planning with diffusion for flexible behavior synthesis. In *International Conference on Machine Learning*, 2022. 3

[22] Zahra Kadkhodaie and Eero Simoncelli. Stochastic solutions for linear inverse problems using the prior implicit in a denoiser. *Advances in Neural Information Processing Systems*, 34:13242–13254, 2021. 3

[23] Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-based generative models. *arXiv preprint arXiv:2206.00364*, 2022. 2, 3, 4

[24] Bahjat Kawar, Michael Elad, Stefano Ermon, and Jiaming Song. Denoising diffusion restoration models. *arXiv preprint arXiv:2201.11793*, 2022. 2, 3

[25] Siddhesh Khandelwal, William Qi, Jagjeet Singh, Andrew Hartnett, and Deva Ramanan. What-if motion prediction for autonomous driving. *arXiv preprint arXiv:2008.10587*, 2020. 3

[26] Jiachen Li, Fan Yang, Hengbo Ma, Srikanth Malla, Masayoshi Tomizuka, and Chiho Choi. Rain: Reinforced hybrid attention inference network for motion forecasting. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 16096–16106, 2021. 3

[27] Wenjie Luo, Cheolho Park, Andre Cornman, Benjamin Sapp, and Dragomir Anguelov. Jfp: Joint future prediction with interactive multi-agent modeling for autonomous driving. In *Conf. On Robot Learning*, 2022. 3, 7

[28] Yecheng Jason Ma, Jeevana Priya Inala, Dinesh Jayaraman, and Osbert Bastani. Diverse sampling for normalizing flow based trajectory forecasting. *arXiv preprint arXiv:2011.15084*, 7(8), 2020. 3

[29] Wei Mao, Miaomiao Liu, and Mathieu Salzmann. Generating smooth pose sequences for diverse human motion prediction. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 13309–13318, 2021. 3

[30] Xiaoyu Mo, Zhiyu Huang, and Chen Lv. Multi-modal interactive agent trajectory prediction using heterogeneous edge-enhanced graph attention network. In *Workshop on Autonomous Driving, CVPR*, volume 6, page 7, 2021. 7

[31] Nigamaa Nayakanti, Rami Al-Rfou, Aurick Zhou, Kratarth Goel, Khaled S Refaat, and Benjamin Sapp. Wayformer: Motion forecasting via simple & efficient attention networks. *arXiv preprint arXiv:2207.05844*, 2022. 3, 4, 7

[32] Jiquan Ngiam, Benjamin Caine, Vijay Vasudevan, Zhengdong Zhang, Hao-Tien Lewis Chiang, Jeffrey Ling, Rebecca Roelofs, Alex Bewley, Chenxi Liu, Ashish Venugopal, David Weiss, Benjamin Sapp, Zhifeng Chen, and Jonathon Shlens. Scene transformer: A unified multi-task model for behavior prediction and planning. *CoRR*, abs/2106.08417, 2021. 3, 7

[33] Alexander Quinn Nichol and Prafulla Dhariwal. Improved denoising diffusion probabilistic models. In *International Conference on Machine Learning*, pages 8162–8171. PMLR, 2021. 2

[34] Geunseob Oh and Huei Peng. Cvae-h: Conditionalizing variational autoencoders via hypernetworks and trajectory forecasting for autonomous driving. *arXiv preprint arXiv:2201.09874*, 2022. 3

[35] Ben Poole, Ajay Jain, Jonathan T Barron, and Ben Mildenhall. Dreamfusion: Text-to-3d using 2d diffusion. *arXiv preprint arXiv:2209.14988*, 2022. 2

[36] Aditya Ramesh, Prafulla Dhariwal, Alex Nichol, Casey Chu, and Mark Chen. Hierarchical text-conditional image generation with clip latents. *arXiv preprint arXiv:2204.06125*, 2022. 2

[37] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10684–10695, 2022. 2, 6

[38] Saeed Saadatnejad, Ali Rasekh, Mohammadreza Mofayezi, Yasamin Medghalchi, Sara Rajabzadeh, Taylor Mordan, and Alexandre Alahi. A generic diffusion-based approach for 3d human pose prediction in the wild. *arXiv preprint arXiv:2210.05669*, 2022. 3

[39] Chitwan Saharia, William Chan, Saurabh Saxena, Lala Li, Jay Whang, Emily Denton, Seyed Kamyar Seyed Ghasemipour, Burcu Karagol Ayan, S Sara Mahdavi, Rapha Gontijo Lopes, et al. Photorealistic text-to-image diffusion models with deep language understanding. *arXiv preprint arXiv:2205.11487*, 2022. 2

[40] Benjamin Sapp, Yuning Chai, Mayank Bansal, and Dragomir Anguelov. Multipath: Multiple probabilistic anchor trajectory hypotheses for behavior prediction. In *Conference on Robot Learning*, pages 86–99. PMLR, 2020. 3

[41] Christoph Schöller and Alois Knoll. Flomo: Tractable motion prediction with normalizing flows. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 7977–7984. IEEE, 2021. 3

[42] Haoran Song, Wenchao Ding, Yuxuan Chen, Shaojie Shen, Michael Yu Wang, and Qifeng Chen. Pip: Planning-informed trajectory prediction for autonomous driving. In *European Conference on Computer Vision*, pages 598–614. Springer, 2020. 3

[43] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. *arXiv preprint arXiv:2010.02502*, 2020. 2, 3

[44] Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456*, 2020. 2, 3

[45] Qiao Sun, Xin Huang, Junru Gu, Brian C Williams, and Hang Zhao. M2i: From factored marginal trajectory prediction to interactive prediction. *arXiv preprint arXiv:2202.11884*, 2022. 3, 7

[46] Charlie Tang and Russ R Salakhutdinov. Multiple futures prediction. In *NeurIPS*. 2019. 3

[47] Ekaterina I. Tolstaya, Reza Mahjourian, Carlton Downey, Balakrishnan Varadarajan, Benjamin Sapp, and Dragomir Anguelov. Identifying driver interactions via conditional behavior prediction. In *IEEE International Conference on Robotics and Automation, ICRA 2021, Xi'an, China, May 30 - June 5, 2021*, pages 3473–3479. IEEE, 2021. 3

[48] Balakrishnan Varadarajan, Ahmed Hefny, Avikalp Srivastava, Khaled S Refaat, Nigamaa Nayakanti, Andre Cornman, Kan Chen, Bertrand Douillard, Chi Pang Lam, Dragomir Anguelov, et al. Multipath++: Efficient information fusion and trajectory aggregation for behavior prediction. *arXiv preprint arXiv:2111.14973*, 2021. 3, 6, 7

[49] Ruihan Yang, Prakhar Srivastava, and Stephan Mandt. Diffusion probabilistic modeling for video generation. *arXiv preprint arXiv:2203.09481*, 2022. 2

[50] Wenyuan Zeng, Wenjie Luo, Simon Suo, Abbas Sadat, Bin Yang, Sergio Casas, and Raquel Urtasun. End-to-end interpretable neural motion planner. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8660–8669, 2019. 3

[51] Wenyuan Zeng, Shenlong Wang, Renjie Liao, Yun Chen, Bin Yang, and Raquel Urtasun. Dsdnet: Deep structured self-driving network. In *European conference on computer vision*, pages 156–172. Springer, 2020. 3

[52] Mingyuan Zhang, Zhongang Cai, Liang Pan, Fangzhou Hong, Xinying Guo, Lei Yang, and Ziwei Liu. Motiondiffuse: Text-driven human motion generation with diffusion model. *arXiv preprint arXiv:2208.15001*, 2022. 3

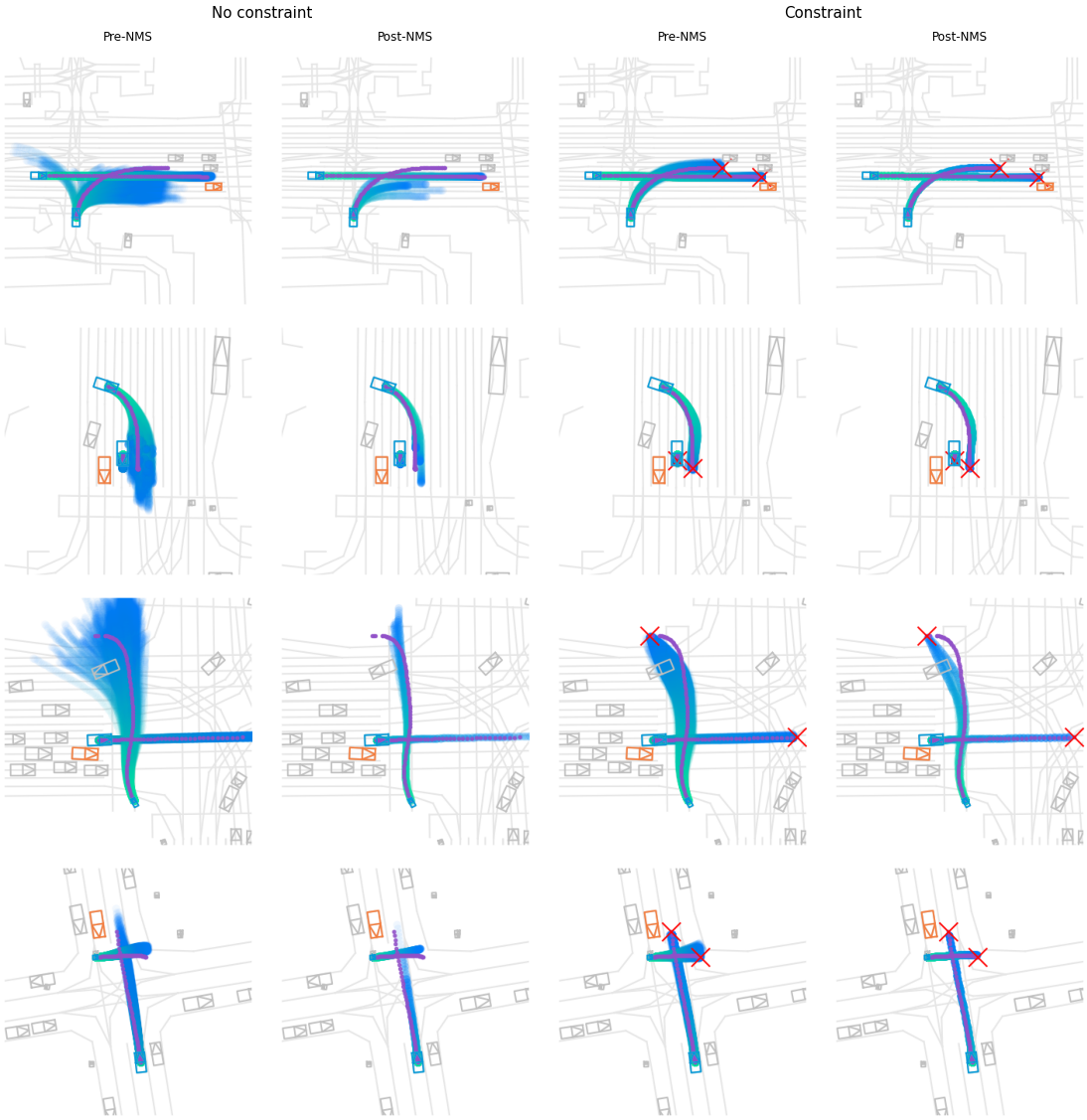
[53] Ziyuan Zhong, Davis Rempe, Danfei Xu, Yuxiao Chen, Sushant Veer, Tong Che, Baishakhi Ray, and Marco Pavone. Guided conditional diffusion for controllable traffic simulation. *arXiv preprint arXiv:2210.17366*, 2022. 3, 7

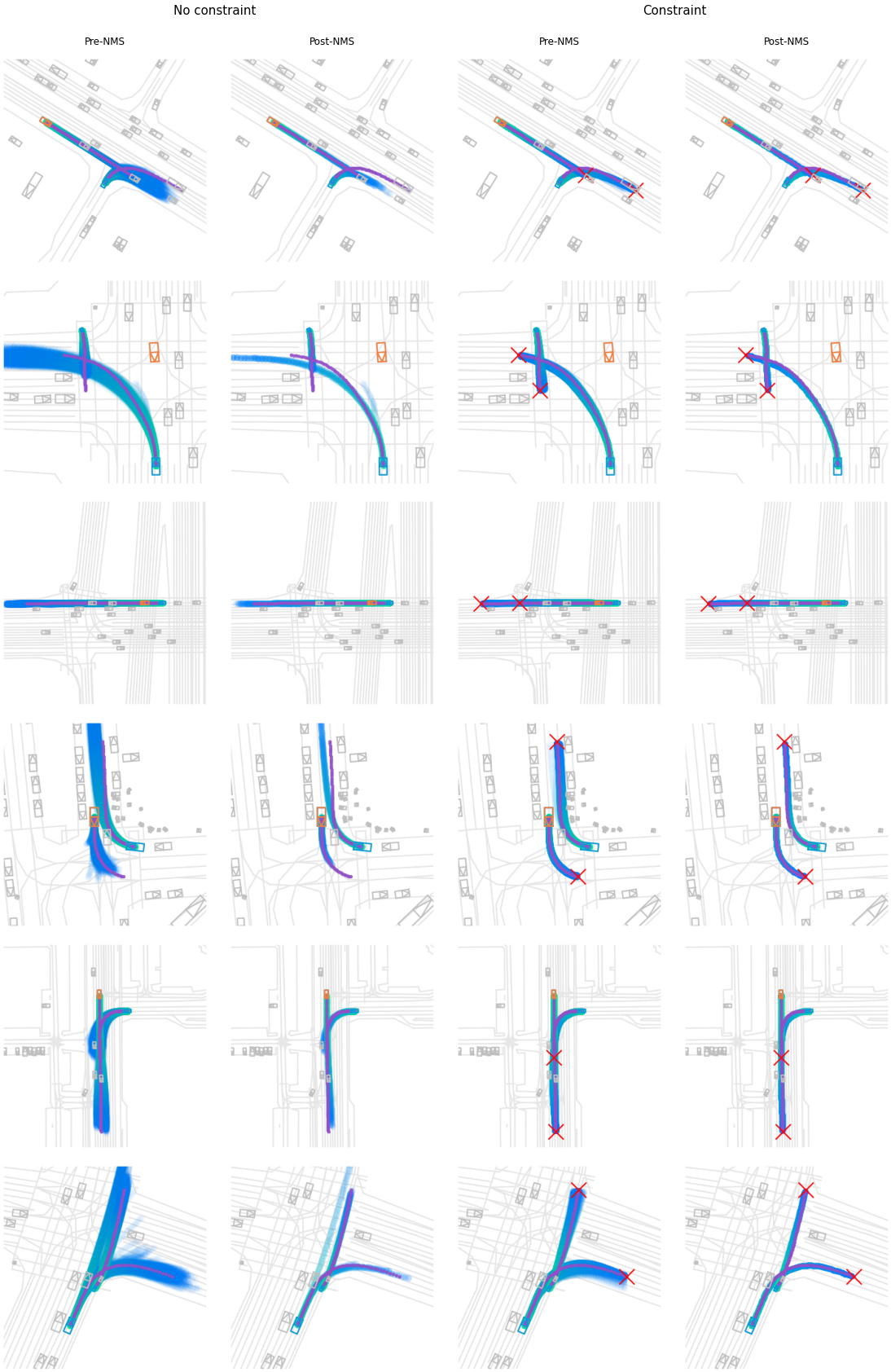
arXiv:2306.03083v1 [cs.RO] 5 Jun 2023

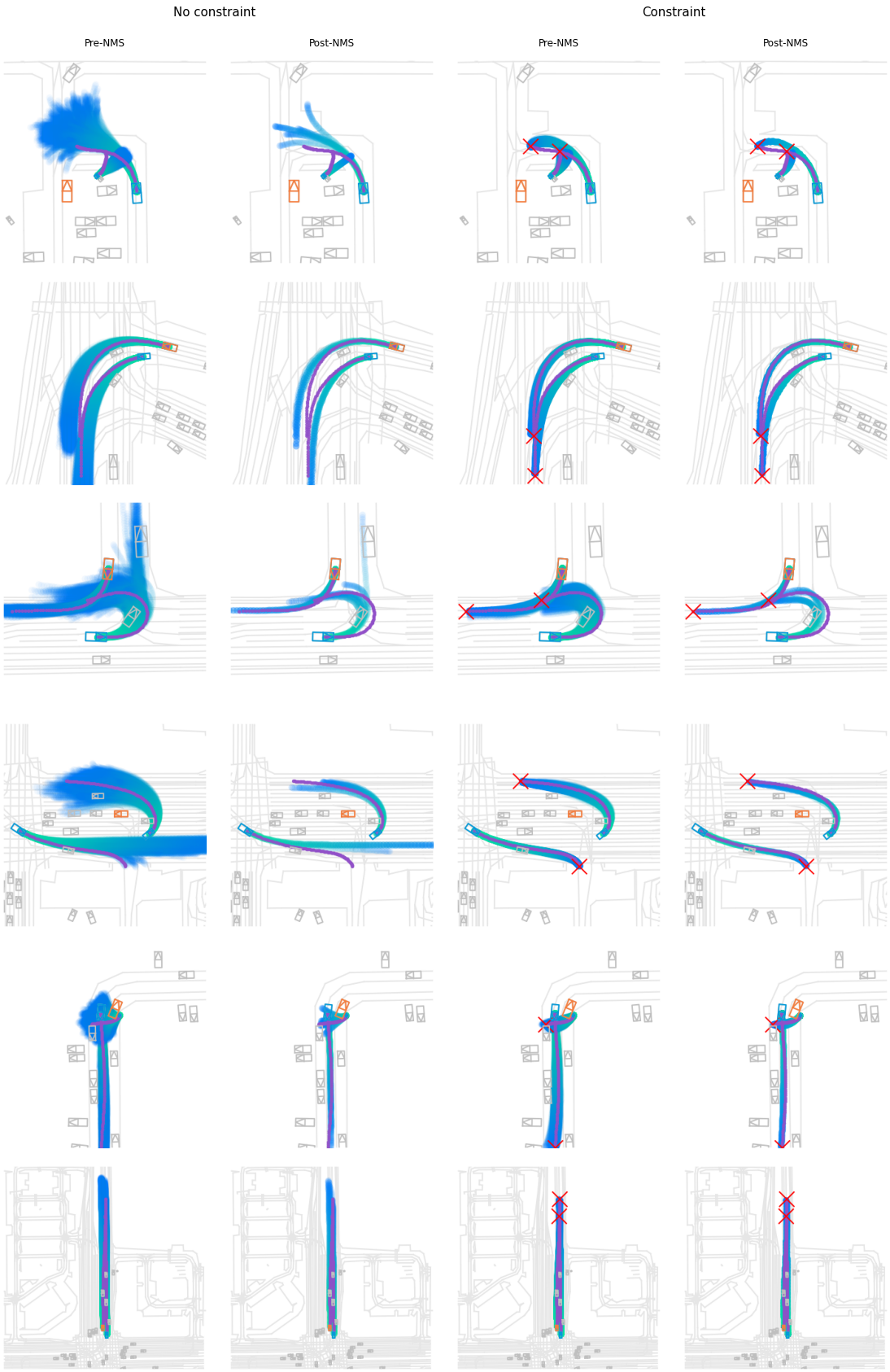
MotionDiffuser: Controllable Multi-Agent Motion Prediction using Diffusion

Chiyu "Max" Jiang* Andre Cornman* Cheolho Park
Ben Sapp Yin Zhou Dragomir Anguelov
* equal contribution
Waymo LLC

1. Additional Visualizations




No constraint
Constraint
Pre-NMS
Post-NMS
Pre-NMS
Post-NMS


No constraint
Constraint
Pre-NMS
Post-NMS
Pre-NMS
Post-NMS

2. Implementation Details

MotionDiffuser is trained on the Waymo Open Motion Dataset using 32 TPU shards for $2 * 10^6$ training steps. We use the ADAMW optimizer [**?**] with weight decay coefficient of 0.03. The learning rate is set to $5 * 10^{-4}$, with 10^4 warmup steps and linear learning rate decay. MotionDiffuser uses the Wayformer [**?**] encoder backbone, with 128 latent embeddings, each with hidden size of 256. Because the Wayformer encoder is agent centric, we append each agent's position and heading (relative to the ego vehicle) to its corresponding context vectors.

Our transformer denoiser architecture uses 4 layers of self-attention and cross-attention blocks. Each attention layer has a hidden size of 256 and an intermediate size of 1024. ReLU activation is used in all transformer layers. We embed the noise level using 128 random fourier features.

We can flexibly denoise N random noise vectors during training and inference. We use $N = 128$ during training and $N = 256$ during inference (before applying clustering).

3. Network Preconditioning

We follow the network preconditioning framework from [**?**], which defines the denoiser $D_{\boldsymbol{\theta}}$ as:

$$D_{\boldsymbol{\theta}}(\boldsymbol{x}; \boldsymbol{c}, \sigma) = c_{\text{skip}}(\sigma)\boldsymbol{x} + c_{\text{out}}(\sigma)F_{\boldsymbol{\theta}}(c_{\text{in}}(\sigma)\boldsymbol{x}; \boldsymbol{c}, c_{\text{noise}}(\sigma)) \tag{1}$$

$c_{\text{in}}(\sigma)$ scales the network input, such that the training inputs to $F_{\boldsymbol{\theta}}$ have unit variance.

$$c_{\text{in}}(\sigma) = 1/\sqrt{\sigma^2 + \sigma_{data}^2} \tag{2}$$

$c_{\text{skip}}(\sigma)$ modulates the skip connection and is defined as:

$$c_{\text{skip}}(\sigma) = \sigma_{data}^2/(\sigma^2 + \sigma_{data}^2) \tag{3}$$

$c_{\text{out}}(\sigma)$ modulates the network output and is defined as:

$$c_{\text{out}}(\sigma) = \sigma \cdot \sigma_{data}/\sqrt{\sigma^2 + \sigma_{data}^2} \tag{4}$$

Finally $c_{\text{noise}}(\sigma)$ scales the noise level, and is defined as:

$$c_{\text{noise}}(\sigma) = \frac{1}{4}\ln\sigma \tag{5}$$

For all our experiments, we set $\sigma_{data} = 0.5$.

4. Inference Latency

We report our model's inference latency over a varying number of sampling steps T in Table 1. We use a single V100 GPU, with batch size of 1.

Method	Latency (ms)	minSADE(↓)	minSFDE(↓)	SMissRate(↓)
Ours ($T = 8$)	101.0	0.91	2.06	0.47
Ours ($T = 16$)	203.7	0.88	1.96	0.44
Ours ($T = 32$)	408.5	0.88	1.97	0.43

Table 1. Model inference latency vs. quality for WOMD Interactive Validation Split.